

TECHNICAL ASSIGNMENT

AI/ML ENGINEER: INFERENCE OPTIMIZATION

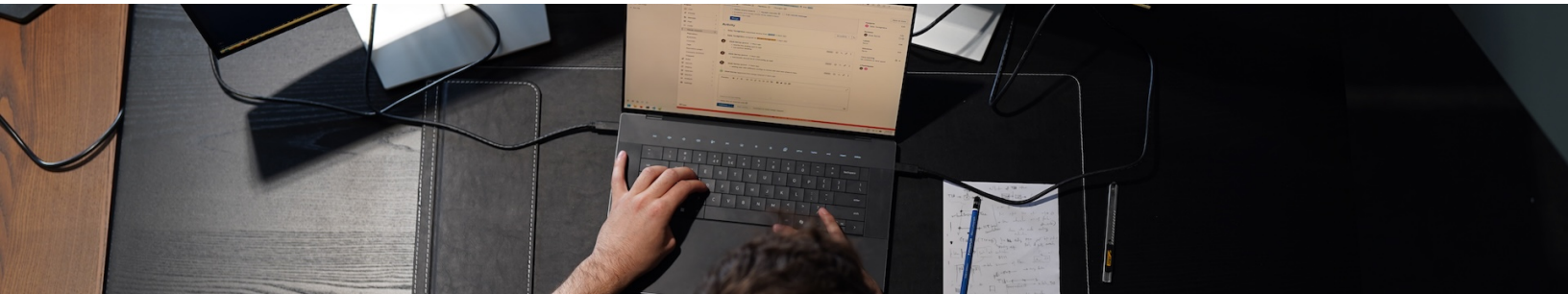


Table of Contents

1	Introduction	2
2	LLM inference on a specialized hardware accelerator	2
2.1	Problem	2
2.2	Task requirements	4
3	Expectations	4
4	Submission Requirements	4
5	Conclusion	5

1 Introduction

As part of our recruitment process for the AI/ML Inference team, tasked with driving the technical vision for Arago's AI accelerator, we invite you to complete the technical assignment below. These tasks are designed to assess your abilities and reasoning when faced with technical challenges.

While the topics of the tasks may not exactly match your past experience, please note that this is intentional. Arago is working with many new design concepts, and this approach allows us to evaluate how you tackle and solve problems outside your comfort zone.

Important Notes:

- The tasks may intentionally involve unfamiliar concepts to simulate our work with cutting-edge AI accelerator designs.
- Focus on demonstrating your problem-solving process rather than delivering perfect solutions.
- Clear reasoning and documentation are as valuable as technical results.

2 LLM inference on a specialized hardware accelerator

2.1 Problem

This task considers an AI inference accelerator with a fixed architecture, consisting of the following components:

- **Host interface:** connects the accelerator to a host CPU (not shown), running a Linux-based operating system. The host interface allows host-DRAM and DRAM-host transfers. The host interface also forwards commands from the host to the accelerator's control CPU.
- **Control CPU:** controls the accelerator's operations and manages the accelerator's resources. It receives kernel launch commands from the host interface and orchestrates the accelerator's resources for the execution of the kernel.
- **DRAM:** is the main memory of the accelerator. It is used to store the model weights and the input data, and potentially intermediate results. It is not directly accessible by the accelerator's compute units, but rather through the DMA engine.
- **DMA engine:** transfers data between the DRAM and the scratchpad SRAM.
- **Scratchpad SRAM:** the main on-chip memory of the accelerator, fully software-managed, used to store the input and output tiles for the systolic arrays.
- **Systolic array 16x16:** given two input tiles A^T and B of $16 \times K$ elements each in the scratchpad SRAM, for K up to 256, the systolic array computes the matrix product $A^T B$ and stores the output tile C (16×16 elements) in the scratchpad SRAM.
- **Systolic array 32x16:** given two input tiles A^T and B of $32 \times K$ elements and $16 \times K$ elements respectively in the scratchpad SRAM, for K up to 256, the systolic array computes the matrix product $A^T B$ and stores the output tile C (32×16 elements) in the scratchpad SRAM.
- **Vector CPU:** generic SIMD CPU core, with low-latency access to the scratchpad SRAM, which is used to perform element-wise, reduction and special operations.

The architecture is illustrated in Figure 1.

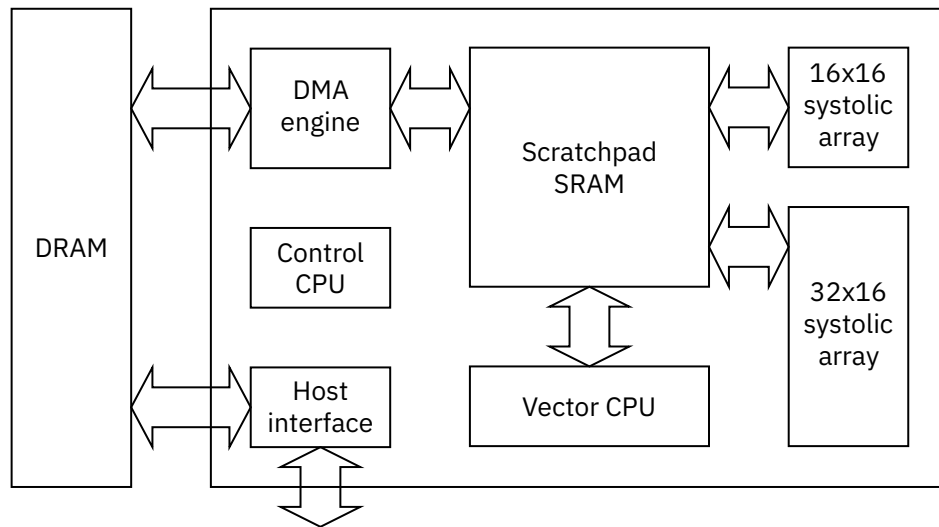


Figure 1: Accelerator architecture.

Parameter	Value
DRAM capacity	64GiB
Scratchpad SRAM capacity	16MiB
Systolic array input data width	8 bits
Systolic array output data width	16 bits
Control CPU to DMA engine configuration latency	200 cycles
Control CPU to vector CPU communication latency	300 cycles
Control CPU to systolic array communication latency	100 cycles
DRAM <-> DMA bandwidth	512 bit/cycle (bidirectional)
DRAM read latency (time to first byte)	200 cycles
DRAM write latency (time to first byte)	150 cycles
DMA <-> SRAM bandwidth	512 bit/cycle (bidirectional)
SRAM <-> vector CPU bandwidth	1024 bit/cycle (bidirectional)
SRAM <-> 16x16 systolic array bandwidth	512 bit/cycle (inputs), 64 bit/cycle (outputs)
SRAM <-> 32x16 systolic array bandwidth	768 bit/cycle (inputs), 64 bit/cycle (outputs)

Moreover, the following is assumed:

- The systolic arrays are pipelined, meaning that two input tiles can be processed as the output tile is being read out.
- The vector CPU is assumed to have infinite computation power (only memory bound).
- The accelerator is assumed to have a single clock domain running at 1 GHz.
- You may ignore bus contention and bank conflicts on the scratchpad SRAM to simplify the analysis.

The workload of interest to run on this accelerator is defined below

Model	Llama 3.1 8B
Weights quantization	8-bit
Activation quantization	16-bit
Input sequence length	2048 tokens
Output sequence length	256 tokens
Batch size	16

2.2 Task requirements

1. Provide a pseudocode implementation of an optimized flash-attention kernel for this architecture, detailing the SRAM allocation strategy, the tiling strategy and the overall computation dataflow. Keep in mind that this code, once implemented, would be executed on the control CPU.
2. Identify and explain the bottleneck of this accelerator's architecture for the flash-attention kernel, i.e. the parameter with the largest elasticity with respect to time required for this kernel to complete.
3. Provide estimations of the TTFT (time to first token) and the interactivity (token generation rate) for this workload and architecture. These numbers must be justified by a performance model which takes into account the architectural constants detailed above.
4. Suggest 2 architectural improvements to the accelerator that would improve the interactivity.
5. Suppose that 4 such accelerators can be connected to a single host CPU.
 - How would you parallelize the computation of this workload across the 4 accelerators?
 - What is the theoretical maximum interactivity that can be achieved by this system?
 - What is the minimum required data bandwidth between the host CPU and the 4 accelerators to achieve 70% of the theoretical maximum interactivity?

3 Expectations

For this technical assignment, you are expected to:

- Apply sound engineering principles and document your process clearly.
- Demonstrate your problem-solving approach and technical reasoning.
- Provide well-documented solutions with clear explanations of your choices.
- Focus on methodology and approach rather than perfect solutions.
- Complete the assignment within a reasonable timeframe (ideally close to a week).

4 Submission Requirements

- Provide a comprehensive report documenting your approach, methodology, and results.
- Include all relevant code, schematics, simulations, or other technical materials. If applicable, please provide build instructions in a README.md.
- Clearly document any assumptions made during the design process.

- Demonstrate a clear understanding of the problem and a methodical approach to solving it.
- Package all materials in a single ZIPped folder with clear structure.
- Book a meeting using this [link](#) and share your results at least **24h before** the selected time slot.

5 Conclusion

This assignment is an opportunity for you to showcase your technical abilities and problem-solving approach. While the tasks are designed to be completed in a short period, they offer a glimpse into the kind of challenges you may face in this role. We encourage you to approach the tasks thoughtfully, leveraging your experience and knowledge to develop innovative and effective solutions.

The tasks described above may depart slightly from your field of expertise or the job description for which you are applying/considered. If so, this is intentional. We want to see how you approach these problems, and what you can deliver in a short amount of time. We are not looking for production-grade solutions; instead, we focus on your problem-solving process, technical reasoning, and ability to adapt to new challenges.

Good luck with your assignment! We look forward to your innovative solutions and insights!